

Taller 4 - Sintaxis y herramientas

Sistemas Digitales

Segundo Cuatrimestre 2026

Guía de referencia de SystemVerilog y del flujo de simulación y síntesis.

1. De la clase a este taller

La clase 3 (secuenciales) es el mapa. Acá se construye en SystemVerilog lo que ahí se dibujó: flip-flop D, enable, registro de N bits, copiar entre registros. La clase no enseña HDL; esta guía es el puente.

En la clase	En el código de este taller
Q, Q_{prev}	q ahora / q si no hubo flanco
flanco ascendente del clock	<code>always_ff @(posedge clk)</code>
FF D con enable (mux a D)	mux: dato nuevo o realimentar Q
wren (write enable)	we (control de escritura, no es un reloj)
rst_s (reset síncronico)	if adentro de <code>always_ff</code>
rst_a (reset asíncrono)	no se usa; la tabla de clase lo muestra
en_i + Hi-Z (quién habla en el bus)	mux: un dato entre varios
Register file / memoria $M \times N$	se dibujó en clase; no entra acá
FSM, Moore, Mealy	clase 3B / Taller 4

En clase el bus compartido se arma con buffers de 3 estados (Hi-Z). En HDL de este taller **no** se usa `inout` ni `1'bz`: un mux elige un dato entre varios. Dos escrituras en el mismo ciclo guardan el mismo valor (no es un cortocircuito).

2. Módulos, puertos y assign

Un módulo tiene interfaz (puertos) y cuerpo (lógica). Las entradas entran, las salidas se asignan, las señales internas conectan subexpresiones.

```
module mayor (  
    input  logic a,  
    input  logic b,  
    output logic y  
);  
    logic p;  
    assign p = a & b; // AND: 1 solo si ambos son 1  
    assign y = p | ~a; // OR y NOT  
endmodule
```

`assign` describe un cable: la salida sigue a la expresión en todo momento.

Lógica	SV	Nombre	Notas
$X \wedge Y$	$x \& y$	AND	1 solo si ambos son 1
$X \vee Y$	$x y$	OR	1 si alguno es 1
$\overline{X}$	$\sim x$	NOT	invierte el bit
$X \oplus Y$	$x \wedge y$	XOR	1 si los bits difieren

XOR se puede reescribir con AND, OR y NOT:

$$X \oplus Y = (X \wedge \overline{Y}) \vee (\overline{X} \wedge Y)$$

3. Buses

Un bit se declara `logic x`. Un grupo de bits, con rango:

```
logic [3:0] a; // 4 bits: a[3] a[2] a[1] a[0]
```

[3:0]: el índice alto a la izquierda [3] es bit más significativo o MSB; [0] es el bit menos significativo o LSB.

Literales con ancho y base:

```
4'b0111 // 4 bits en binario: literalmente los bits `0111`
```

```
1'b0 // un bit en 0
```

```
5'd1 // 5 bits en decimal: los 5 bits que representan el 1, que son 00001
```

&, |, ^ y ~ también operan **bit a bit** sobre un bus: ~a invierte los 4 bits; a ^ b hace XOR de cada par a[i], b[i].

4. Concatenación y replicación

Llaves juntan bits. {3{en}} copia en tres veces: {en, en, en}.

```
assign word = {hi, lo}; // dos buses (o bits) pegados
```

```
assign copies = {3{en}}; // {en, en, en}
```

5. Condicional (multiplexor)

sel ? a : b vale a si sel es 1, y b si es 0. Es un mux de 2 a 1: elige entre dos cables según un bit de control.

```
assign out = sel ? e_1 : e_0; // sel=1 → out=e_1; sel=0 → out=e_0
```

También opera sobre buses: ctrl ? resta : suma elige uno de los dos resultados.

Se pueden anidar: sel_hi ? a : (sel_lo ? b : c).

6. Cómo conectar módulos

Instanciar es poner una **copia** de un módulo adentro de otro y cablear puertos. Cada copia tiene un nombre de instancia distinto.

Ejemplo: Dado el módulo xor2, que hace XOR de dos bits, así se implementaría el módulo paridad3, que verifica la paridad de tres entradas, y utiliza el módulo xor2 en su implementación.

```
module xor2 (
    input logic a,
    input logic b,
    output logic y
```

```

);
    assign y = a ^ b;
endmodule

module paridad3 (
    input  logic x,
    input  logic y,
    input  logic z,
    output logic f
);
    logic p;

    xor2 u0 ( // llamaremos u0 a esta copia del módulo xor2
        .a(x), // al input a de u0, le conectamos x (el 1er input de paridad3)
        .b(y), // al input b de u0, le conectamos y (el 2do input de paridad3)
        .y(p)  // al output y de u0, lo conectamos nuestro auxiliar p (cableamos el
output de u0 a p)
    );

    xor2 u1 ( // llamaremos u1 a esta segunda copia del módulo xor2
        .a(p), // al input a de u1, le conectamos p, que tiene la salida de u0
        .b(z), // al input b de u1, le conectamos z (el 3er input de paridad3)
        .y(f)  // al output y de u1, lo conectamos a la salida f (salida del módulo
paridad3)
    );
endmodule

```

- xor2 es el tipo de módulo. u0 y u1 son los apodos de las dos copias.
- la sintaxis .a(x) conecta el puerto a del módulo que se está instanciando con la señal x que vive dentro del módulo padre.
- logic p es un cable interno: sale de u0 y entra a u1.

Conexión por nombre, no posicional. Si se mezclan los puertos, el circuito compila y está mal.

7. Cómo traer un módulo de otro archivo

Instanciar usa el **nombre** del module. Ese module tiene que estar en algún .sv que el compilador vea.

No se editan copias a mano. El Makefile lista los .sv de ejercicios anteriores. make sim (o make deps) los **copia** a la carpeta actual, pisando copias viejas. La fuente de verdad es el ejercicio de origen: hay que resolver en orden.

HDL Studio no lee el Makefile y *Create circuit* suma solo el archivo abierto. Por eso la copia: los .sv quedan al lado del módulo nuevo. Hay que **agregarlos al circuito** (clic derecho → *Add to HDL Studio*, o seleccionar todos los .sv salvo el _tb y *Create circuit*). Sin ese paso, Yosys dice **unknown module**.

Si se quiere reutilizar un módulo propio extra: copiarlo a la carpeta (o sumarlo a PRE_SRCS del Makefile) y agregarlo al circuito de HDL Studio.

``include` pega un archivo en SystemVerilog. Verilator lo sigue. HDL Studio no: copia los fuentes a un temporal. Para sintetizar, el .sv tiene que estar **agregado al circuito**.

SystemVerilog también tiene **import** para **packages**. Acá los módulos se reutilizan por instancia + archivos en la carpeta.

8. Reloj y `always_ff`

En Taller 1 todo era combinacional: la salida es función de las entradas **ahora**. Un registro tiene memoria: su salida `q` depende de lo que se guardó en un **flanco** del reloj.

El reloj `clk` es una señal periódica. El **flanco ascendente** (`posedge`) es el instante en que `clk` pasa de 0 a 1. En ese instante (y solo ahí) el flip-flop mira `d` y actualiza `q`.

```
always_ff @(posedge clk) begin
    q <= d;
end
```

- `always_ff` le dice al sintetizador: esto es un flip-flop, no un cable.
- `@(posedge clk)` es la lista de sensibilidad: el bloque corre en cada flanco ascendente de `clk`.
- Entre flancos, `q` **conserva** su valor. Cambiar `d` a mitad de ciclo no mueve `q`.

`clk` y `rst` son señales de **control**, no de dato. No se usa `clk` adentro de un `assign` para “fabricar” otro reloj.

No escribir `always @(posedge clk)` ni `always @(*)`. En este taller el bloque de estado es `always_ff`.

9. Asignación no bloqueante `<=`

Adentro de `always_ff` se usa `<=`, no `=`.

```
q <= d; // en este flanco, q pasará a valer lo que d vale ahora
```

`<=` significa: anotá el valor nuevo y aplicalo al **final** del flanco. Si hay varios `<=` en el mismo bloque, todos leen los valores **viejos** y después actualizan juntos.

`assign y = a & b` sigue siendo el cable combinacional. No se usa `assign` para describir memoria: un `assign q = d` sería un cable, no un flip-flop.

Dónde	Símbolo	Hardware
<code>assign</code> (Taller 1)	<code>=</code>	cable / compuerta
<code>always_ff</code>	<code><=</code>	flip-flop: guarda en el flanco

Usar `=` adentro de `always_ff` está mal en este taller. El linter se queja y el simulador puede mostrar un valor que el circuito real no tiene.

10. `if` / `else if` / `else`

En Taller 1 la decisión era el mux ? ∴ Acá, adentro de `always_ff`, se escribe la **prioridad** con `if`.

```
always_ff @(posedge clk) begin
    if (boton_izq)
        pos <= 2'b00; // si boton_izq es 1, esta rama gana
    else if (boton_der)
        pos <= 2'b11; // si no, y boton_der es 1
    // si no entra a ningún if: pos no se toca → conserva
end
```

Se lee de arriba hacia abajo. La primera condición verdadera es la que cuenta.

No hace falta un `else` final que copie `pos <= pos`. Si una rama no asigna `pos`, el flip-flop conserva.

`if (boton_izq)` trata la señal como condición: 1 es verdadero, 0 es falso. No hace falta escribir `if (boton_izq == 1'b1)`.

11. begin / end

Un `if` o un `always_ff` admite **una** sentencia. Si hay que hacer más de una cosa, se agrupan con `begin ... end` (como llaves en otros lenguajes).

```
always_ff @(posedge clk) begin
    if (start) begin
        a <= 1'b1;
        b <= 1'b0;
    end else if (stop) begin
        a <= 1'b0;
        b <= 1'b1;
    end
end
```

Si se olvidan, solo la primera línea queda adentro del `if`. El resto se ejecuta siempre. Poner `begin/end` en el `always_ff` evita ese error.

12. El literal '0

`'0` pone todos los bits en 0, del ancho que haga falta. Evita contar ceros a mano.

```
q <= 1'b0;    // un bit
q <= 4'b0000;
q <= '0;      // el mismo 0, del ancho de q
```

En este taller se puede usar `'0` o `4'b0000`. No mezclar anchos a ojo: `1'b0` en un bus de 4 bits se extiende, pero el literal con ancho es más claro.

13. Qué no usar (todavía)

No	Por qué
<code>always @(posedge clk)</code>	usar <code>always_ff</code>
<code>=</code> adentro de <code>always_ff</code>	usar <code><=</code>
<code>assign q = d</code> para un registro	eso es un cable, no memoria
<code>always_comb</code> , <code>case</code> , <code>enum</code>	clase B / Taller 3 (FSM)
<code>inout</code> , <code>1'bz</code> , <code>Hi-Z</code>	en clase el bus; acá se usa un mux
<code>latch</code> / <code>always_latch</code> / FF JK	concepto de clase, no ejercicio
<code>for</code> , <code>generate</code>	igual que Taller 1
usar <code>clk</code> en un <code>assign</code>	no fabricar clocks
<code>always_ff</code> disparado por <code>we</code>	<code>we</code> es control, no reloj

Una señal (`q`) se asigna desde **un solo** bloque. Dos `always_ff` que escriben `q` no son un circuito válido.

14. Cómo correr la simulación

En la carpeta del ejercicio:

```
make sim
```

Si todos los casos pasan, aparece un recuadro verde **PASS**. Si alguno falla, líneas **FAIL: value = ...** y el proceso termina con error.

En este taller el testbench no recorre las 2^N combinaciones de una vez (eso era Taller 1, combinacional). Aplica una **secuencia** de ciclos: reset, después escrituras y lecturas. Por eso **make wave** pasa a ser parte del flujo: hay que ver **clk**, entradas y **q** alineados a los flancos.

```
make wave    # abre la traza en Surfer (después de make sim)
make clean   # borra obj_dir/ y la traza
```

make clean borra **obj_dir/** (el compilado). No hace falta en el flujo normal: **make sim** vuelve a construir.

Los **_tb.sv** son parte de la especificación: se pueden leer, no se editan.

15. Cómo sintetizar

Simular comprueba comportamiento. Sintetizar muestra el **circuito**.

1. En la carpeta del ejercicio: **make sim** (copia los **.sv** anteriores si hace falta).
2. Abrir el **.sv** del módulo nuevo (no el **_tb**) y *Create circuit in HDL Studio*.
3. Agregar al circuito los **.sv** copiados que instancia (ahora están en la misma carpeta). No agregar el **_tb.sv**. Atajo: seleccionar todos los **.sv** menos el **_tb** y *Create circuit*.
4. **Top module**: el módulo nuevo. *Synthesize*.

Si el log dice **unknown module**, falta un **.sv** en el circuito.

En un **always_ff** bien escrito, HDL Studio tiene que mostrar **flip-flops**, no solo compuertas. Si se ve un cable de **d** a **q** sin registro, el **always_ff** está mal (o se usó **assign**).

16. Herramientas y bibliotecas

Nada de esto se instala a mano: vive en el contenedor (**twint/sd**). Esta sección es para tener de referencia, no se requiere aun comprender cada parte.

16.1. Entorno

- **VS Code + Dev Container**. **.devcontainer/devcontainer.json** pide la imagen y las extensiones. Al abrir la carpeta, “Reopen in Container”.
- **HDL Studio** (**twint.hdl-studio**). Extensión de la materia. Toma el SystemVerilog, lo sintetiza y dibuja el circuito interactivo.
- **Verilog HDL** (**mshr-h.veriloghdl**). Coloreo y navegación del código.

16.2. Simulación

- **Verilator** (**verilator.org**). Compila SystemVerilog a C++ y lo ejecuta. **make sim** llama **verilator --binary --timing** con **--top-module sim_top**.
- **Make**. Cada carpeta define **TB**, **SRCS** y a veces **PRE_SRCS**. **make sim** copia **PRE_SRCS** a la carpeta y después llama a Verilator.
- **sim_top.sv**. Genera **clk** y **rst**, instancia el testbench (**TB_MODULE**) y corta la corrida con **\$fatal** si falló algún caso.

- **Surfer.** Visor de formas de onda. `make wave` abre el dump (FST/VCD) que dejó la simulación. En secuencial: identificar flancos de `clk` y mirar **d antes** del flanco y **q después**.

No hace falta escribir testbenches en este taller. Se pueden leer para ver en qué ciclo el TB cambia las entradas.

16.3. Síntesis y lenguaje

- **Yosys** (yosyshq.net/yosys). Sintetizador de código abierto. HDL Studio lo usa (`read_slang`) para inferir compuertas y flip-flops a partir de `assign`, `always_ff` e instancias. Copia al directorio temporal **solo** los `.sv` agregados al circuito.
- **SystemVerilog (IEEE 1800).** El subconjunto de este taller está en esta guía: lo de Taller 1 (`module`, `logic`, `assign`, operadores, `?`, instancias) más `always_ff @(posedge clk)`, `<=`, `if/else if`, `begin/end`, buses `[3:0]` y `'0`. El estándar completo está en IEEE; una introducción amigable es Harris y Harris, **Digital Design and Computer Architecture** (HDL combinacional y secuencial, §§4.2 y 4.4).

16.4. En el contenedor, todavía no vistos en los talleres

- **Slang.** Language server: subrayado de errores mientras se escribe. El frontend `read_slang` de Yosys usa el mismo compilador.
- **Verible.** Formateador y linter (`verible-verilog-format`).
- **`always_comb`, `case`, `typedef enum`.** Lógica combinacional por casos y nombres de estado. Clase B (FSM) y Taller 3.